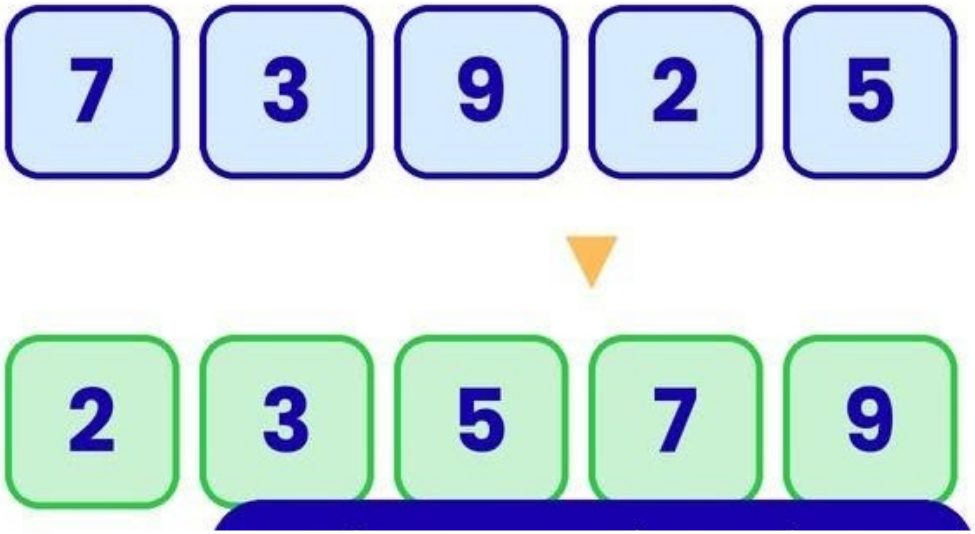
Les algorithmes de tri

Python • Algorithmique



Belgacem DKHIL 4 ème SCEXP

C'est quoi le tri ?

Le tri consiste à réorganiser les éléments d'un tableau selon un ordre déterminé.

Tableau de départ

7	3	9	2	5

Tri croissant
du plus petit au plus grand

2	3	5	7	9

Tri décroissant
du plus grand au plus petit

9	7	5	3	2

Pourquoi trier ? Organiser, lire, rechercher, classer.

Tri à bulles

On compare deux voisins et on les échange s'ils sont mal rangés. Les grandes valeurs remontent comme des bulles.

T = [5, 2, 8, 1]

1) T[0] > T[1] → 5 > 2 : Vrai
On échange



2) T[1] > T[2] → 5 > 8 : Faux
Aucun échange



3) T[2] > T[3] → 8 > 1 : Vrai
On échange



8, la plus grande valeur, est maintenant à la fin.

En répète jusqu'à pas de permutation.

```
procédure Tri_bulle(@T : Tab , n : entier )
début
    répéter
        permut ← faux
        pour i de 0 a n-2 faire
            si T[i] > T[i+1] alors
                aux ⟵ T[i]
                T[i] ⟵ T[i+1]
                T[i+1] ⟵ aux
                permut ← vrai
            fin si
        Fin pour
    jusqu'à (permut = faux)
fin
```

Tri à bulle
Méthode 1

T.D.O.L

Objet	type/nature
permut	booléen
i	entier
aux	type valeurs du tableau T

```
procédure Tri_bulle (@T : Tab , n : entier )
début
    permut ← vrai
    Tant que (permut = vrai) faire
        permut ← faux
        pour i de 0 a n-2 faire
            si T[i] > T[i+1] alors
                aux ← T[i]
                T[i] ← T[i+1]
                T[i+1] ← aux
                permut ← vrai
            fin si
        fin pour
    Fin Tant que
fin
```

Tri à bulle
Méthode 2

T.D.O.L

Objet	type/nature
permut	booléen
i	entier
aux	type valeurs du tableau T

Tri à bulles en Python

```python
def tri_bulles(T, n):
    p = True
    while p == True:
        p = False
        for i in range(0, n-1):
            if T[i] > T[i+1]:
                aux = T[i]
                T[i] = T[i+1]
                T[i+1] = aux
                p = True
```

À retenir : le tri à bulles compare des voisins
et les échange s'ils sont dans le mauvais ordre.

Tri par sélection

On cherche le minimum de la partie non triée,
puis on le place à sa bonne position.

Départ



Étape 1
min = 2, échange avec T[0]



Étape 2
min = 4, déjà bien placé



Étape 3
min = 5, échange avec T[2]



Vert = partie triée Orange = minimum Rose = échange

Tri par sélection

```
procédure Tri_selection(@T : Tab ,n : entier )
début
    pour  i de 0 à n-2 faire
        pmin ⟵  i
        pour j de i+1 à n-1 faire
            si T[j] < T[pmin] alors
                pmin ⟵  j
            fin si
        Fin pour
        si pmin <> i alors
            aux ⟵  T[i]
            T[i] ⟵  T[pmin]
            T[pmin] ⟵  aux
        fin si
    fin pour
fin
```

T.D.O.L

objet	type/nature
p	t
i,j,pmin	entier
aux	type des valeurs du tableau

Tri par sélection : code

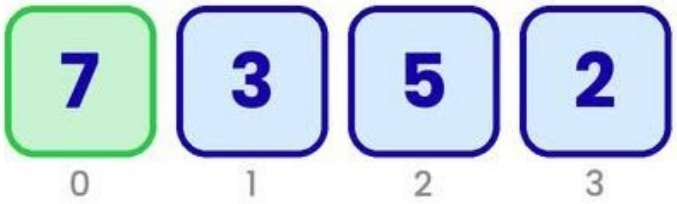
```
1  def tri_select(T, n):
2      for i in range(0, n-1):
3          posmin = i
4          for j in range(i+1, n):
5              if T[j] < T[posmin]:
6                  posmin = j
7          if posmin != i:
8              aux = T[i]
9              T[i] = T[posmin]
10             T[posmin] = aux
```

À retenir : le tri par sélection cherche le minimum dans la partie non triée, puis le place à sa position.

Tri par insertion

Comme des cartes dans ta main : on prend un élément et on l'insère à la bonne place dans la partie triée.

Départ
le 1er élément est déjà trié



On prend 3
on l'insère avant 7



On prend 5
on l'insère entre 3 et 7



On prend 2
on l'insère au début



Tri par Insertion

```
procédure Tri_Insertion(@T : Tab , n : entier )
début
    pour  i de 1 à n-1 faire
        aux ← T[i]
        j ← i
        Tant que (j > 0) et (T[j-1] > T[j]) faire
            T[j] ← T[j-1]
            j ← j-1
        fin Tant que
        T[j] ← aux
    fin pour
fin
```

T.D.O.L

Objet	type/nature
i,j	entier
aux	type valeurs du tableau T

Tri par insertion : code

```python
def tri_insertion(T, n):
    for i in range(1, n):
        aux = T[i]
        j = i
        while j > 0 and T[j-1] > aux:
            T[j] = T[j-1]
            j = j-1
        T[j] = aux
```

À retenir : on construit le tableau trié pas à pas,
en insérant chaque élément à la bonne position.

À retenir

1. **Tri à bulles**
   Compare des voisins et les échange

2. **Tri par sélection**
   Cherche le minimum et le place

3. **Tri par insertion**
   Insère chaque élément à sa place